Clustering -:

Assignment Questions

Theoretical Questions:

1 What is unsupervised learning in the context of machine learning

Unsupervised learning is a type of machine learning in which a model learns patterns, structures, and relationships from **unlabeled data** without any predefined target output. The algorithm analyzes the data to identify hidden patterns, similarities, or groupings on its own. Common tasks in unsupervised learning include **clustering**, **association rule mining**, and **dimensionality reduction**. Examples of unsupervised learning algorithms are **K-Means Clustering**, **Hierarchical Clustering**, and **Principal Component Analysis (PCA)**.

2 How does K-Means clustering algorithm work

K-Means clustering is an unsupervised learning algorithm used to group similar data points into **K clusters**. It works by first selecting K initial centroids and then assigning each data point to the nearest centroid based on distance. After assignment, the centroids are recalculated as the mean of all points in each cluster. This process of assigning points and updating centroids is repeated until the centroids no longer change significantly or a maximum number of iterations is reached, resulting in well-defined clusters.

3 Explain the concept of a dendrogram in hierarchical clustering

A **dendrogram** is a tree-like diagram used in **hierarchical clustering** to visualize how data points or clusters are merged or divided at different levels of similarity. Each branch represents a cluster, and the height at which two branches join indicates the distance or dissimilarity between them. By analyzing the dendrogram, users can determine the appropriate number of clusters by cutting the tree at a chosen height. It helps in understanding the hierarchical relationships among data points and the clustering process.

4 What is the main difference between K-Means and Hierarchical Clustering

The main difference between **K-Means** and **Hierarchical Clustering** is that K-Means requires the number of clusters (**K**) to be specified in advance and partitions the data into K clusters by iteratively updating cluster centroids. In contrast, Hierarchical Clustering does not require a predefined number of clusters and creates a hierarchy of clusters represented by a dendrogram. K-Means is generally faster and suitable for large datasets, while Hierarchical Clustering provides a more detailed view of data relationships but is computationally more expensive.

5 What are the advantages of DBSCAN over K-Means

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) has several advantages over **K-Means**. It does not require the number of clusters to be specified beforehand and can identify clusters of arbitrary shapes, whereas K-Means works best with spherical clusters. DBSCAN is also effective at detecting and handling noise or outliers by labeling them separately, while K-Means assigns every data point to a cluster. These features make DBSCAN more suitable for datasets with irregular cluster structures and noisy data.

6 When would you use Silhouette Score in clustering

The **Silhouette Score** is used to evaluate the quality of clustering by measuring how well each data point fits within its assigned cluster compared to other clusters. It helps determine the optimal number of clusters and assess the effectiveness of a clustering algorithm. The score ranges from **-1 to 1**, where a value close to **1** indicates well-separated clusters, **0** indicates overlapping clusters, and a negative value suggests that some data points may be assigned to the wrong cluster.

7 What are the limitations of Hierarchical Clustering

Hierarchical Clustering has several limitations. It is computationally expensive and requires high memory, making it unsuitable for very large datasets. Once clusters are merged or split, the process cannot be reversed, which may lead to suboptimal clustering results. It is also sensitive to noise and outliers, which can significantly affect the cluster structure. Additionally, choosing the correct level at which to cut the dendrogram to form clusters can be subjective and challenging.

8 Why is feature scaling important in clustering algorithms like K-Means

Feature scaling is important in clustering algorithms like **K-Means** because the algorithm uses distance measures, such as Euclidean distance, to group data points. If features have different scales, variables with larger values can dominate the distance calculation and influence the clustering results disproportionately. By applying scaling techniques such as **Normalization** or **Standardization**, all features contribute equally to the clustering process, leading to more accurate and meaningful clusters.

9 How does DBSCAN identify noise points

DBSCAN identifies noise points based on the density of data points in a region. It uses two parameters: **Eps (ε)**, which defines the neighborhood radius, and **MinPts**, the minimum number of points required to form a dense region. Points that have fewer than MinPts neighbors within the Eps radius and are not reachable from any core point are labeled as **noise (outliers)**. These noise points do not belong to any cluster and are treated separately by the algorithm.

10 Define inertia in the context of K-Means

Inertia in K-Means clustering is a measure of how closely data points are grouped within their assigned clusters. It is calculated as the sum of the squared distances between each

data point and the centroid of its cluster. A lower inertia value indicates that the data points are closer to their centroids, resulting in more compact and well-defined clusters. Inertia is commonly used in the **Elbow Method** to help determine the optimal number of clusters for a dataset.

11 What is the elbow method in K-Means clustering

The **Elbow Method** is a technique used to determine the optimal number of clusters (**K**) in **K-Means clustering**. It involves running the K-Means algorithm for different values of K and calculating the corresponding **inertia** (within-cluster sum of squares). The inertia values are then plotted against K, and the point where the curve starts to bend sharply, forming an “elbow,” is chosen as the optimal number of clusters. This point represents a balance between minimizing inertia and avoiding unnecessary clusters.

12 Describe the concept of "density" in DBSCAN

In **DBSCAN**, **density** refers to the concentration of data points within a specific region of the dataset. A region is considered dense if it contains at least a minimum number of points (**MinPts**) within a specified radius (**Eps**). DBSCAN forms clusters by connecting these dense regions, where points in high-density areas belong to the same cluster. Points located in low-density regions that do not meet the density criteria are treated as noise or outliers.

13 Can hierarchical clustering be used on categorical data

Yes, **Hierarchical Clustering** can be used on **categorical data**, provided an appropriate similarity or distance measure is chosen. Unlike numerical data, where distances such as Euclidean distance are used, categorical data often require measures like **Hamming distance**, **Jaccard similarity**, or other specialized metrics. By using these measures, hierarchical clustering can group similar categorical observations and represent their relationships through a dendrogram.

14 What does a negative Silhouette Score indicate

A **negative Silhouette Score** indicates that a data point is likely assigned to the wrong cluster. It means the point is, on average, closer to points in a neighboring cluster than to points in its own cluster. This suggests poor cluster separation and possible overlap between clusters. Therefore, a negative score is a sign that the clustering result may not accurately represent the underlying structure of the data.

15 Explain the term "linkage criteria" in hierarchical clustering

Linkage criteria in **Hierarchical Clustering** refer to the method used to measure the distance between clusters when deciding which clusters should be merged. Different linkage methods include **single linkage** (minimum distance between points), **complete linkage** (maximum distance), **average linkage** (average distance), and **Ward’s linkage** (minimizes within-cluster variance). The choice of linkage criterion affects the shape and structure of the resulting clusters and dendrogram.

16 Why might K-Means clustering perform poorly on data with varying cluster sizes or densities

K-Means clustering may perform poorly on data with varying cluster sizes or densities because it assumes that clusters are roughly spherical, equally sized, and have similar densities. The algorithm assigns points based on their distance to cluster centroids, which can cause larger or denser clusters to dominate smaller or sparser ones. As a result, data points may be incorrectly assigned, leading to inaccurate cluster boundaries and reduced clustering performance on complex datasets.

17 What are the core parameters in DBSCAN, and how do they influence clustering

The two core parameters in **DBSCAN** are **Eps (ε)** and **MinPts**. Eps defines the maximum distance within which neighboring points are considered part of the same region, while MinPts specifies the minimum number of points required to form a dense cluster. A larger Eps can merge nearby clusters, whereas a smaller Eps may classify more points as noise. Similarly, a higher MinPts requires denser regions to form clusters, making the algorithm more selective and robust to noise.

18 How does K-Means++ improve upon standard K-Means initialization

K-Means++ improves upon standard **K-Means** by selecting initial centroids more strategically rather than choosing them randomly. The first centroid is selected randomly, and subsequent centroids are chosen with a higher probability for points that are far from existing centroids. This approach spreads the initial centroids across the dataset, reducing the chances of poor cluster initialization. As a result, K-Means++ often produces better clustering results, converges faster, and is less likely to get stuck in suboptimal solutions.

19 What is agglomerative clustering

Agglomerative Clustering is a bottom-up hierarchical clustering technique in which each data point starts as an individual cluster. The algorithm repeatedly merges the two most similar or closest clusters based on a chosen linkage criterion until all data points belong to a single cluster or a stopping condition is reached. The clustering process is represented using a dendrogram, which helps visualize the hierarchy of clusters and determine the appropriate number of clusters.

20 What makes Silhouette Score a better metric than just inertia for model evaluation

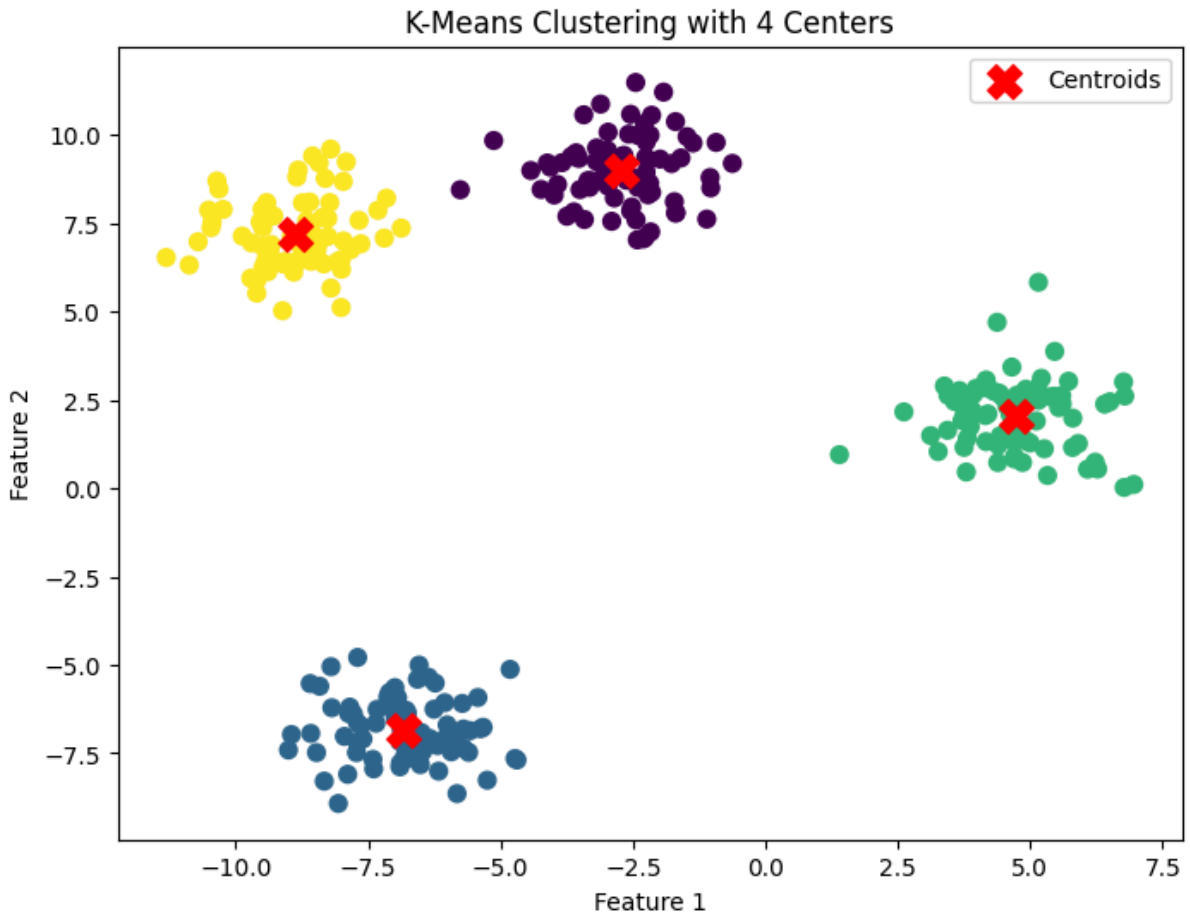
The **Silhouette Score** is often considered a better metric than **inertia** because it evaluates both **cluster compactness** and **cluster separation**. While inertia only measures how close data points are to their cluster centroids and always decreases as the number of clusters increases, the Silhouette Score assesses how well each point fits within its own cluster compared to other clusters. This provides a more balanced and reliable measure of clustering quality and helps in selecting the optimal number of clusters.

Practical Questions:

21 Generate synthetic data with 4 centers using make_blobs and apply K-Means clustering. Visualize using a scatter plot

```
# Generate synthetic data with 4 centers and apply K-Means Clustering

from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Generate synthetic dataset
X, y = make_blobs(n_samples=300,
                  centers=4,
                  cluster_std=1.0,
                  random_state=42)

# Apply K-Means
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X)

# Visualize clusters
plt.figure(figsize=(8,6))
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis', s=50)

# Plot centroids
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            color='red',
            marker='X',
            s=200,
            label='Centroids')

plt.title('K-Means Clustering with 4 Centers')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```

Output -



22 Load the Iris dataset and use Agglomerative Clustering to group the data into 3 clusters. Display the first 10 predicted labels

```
# Load Iris dataset and apply Agglomerative Clustering

from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering

# Load dataset
iris = load_iris()
X = iris.data

# Apply Agglomerative Clustering
agg_clustering = AgglomerativeClustering(n_clusters=3)
labels = agg_clustering.fit_predict(X)

# Display first 10 predicted labels
print("First 10 Predicted Labels:")
print(labels[:10])
```

Output -

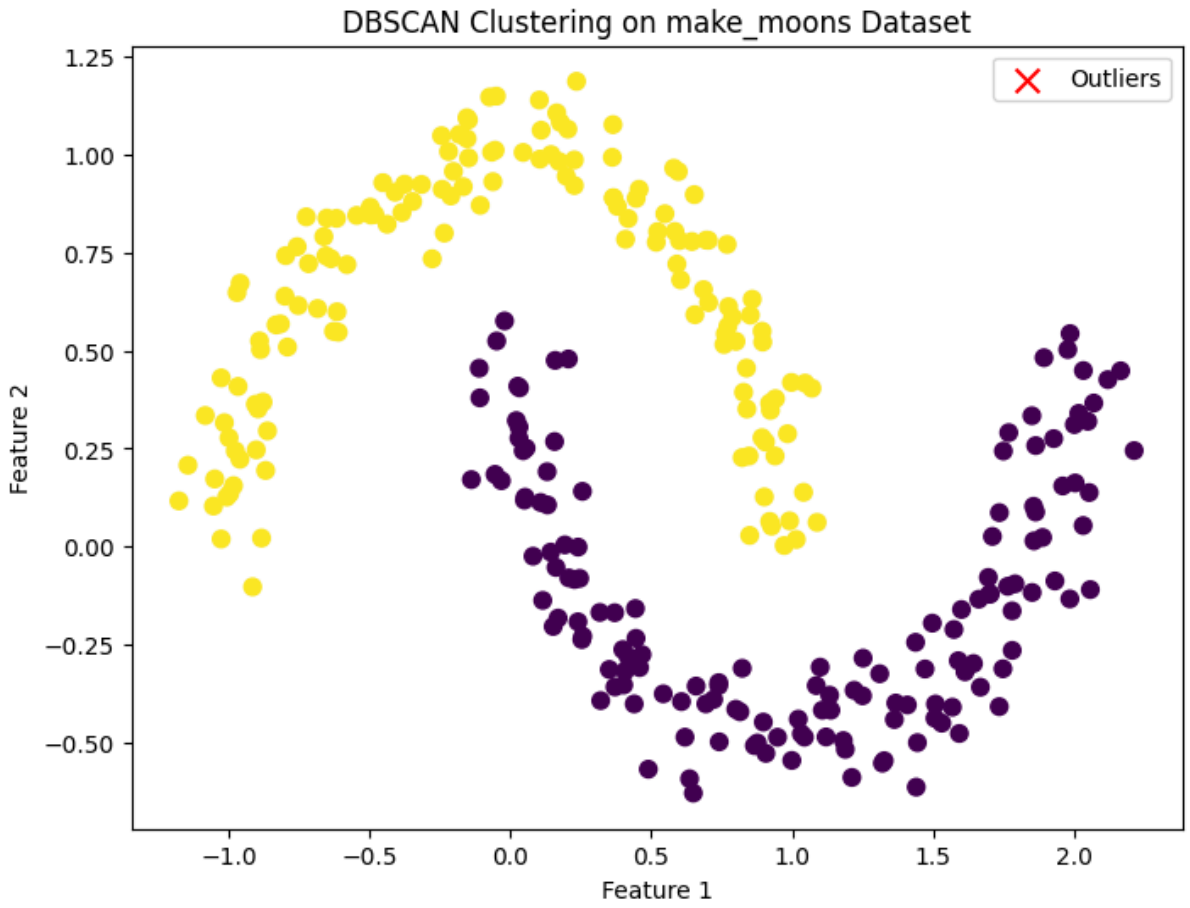
```
First 10 Predicted Labels:
```

```
[1 1 1 1 1 1 1 1 1 1]
```

23 Generate synthetic data using make_moons and apply DBSCAN. Highlight outliers in the plot

```
# Generate synthetic moon-shaped data and apply DBSCAN

from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt
import numpy as np

# Generate dataset
X, y = make_moons(n_samples=300, noise=0.08, random_state=42)

# Apply DBSCAN
dbscan = DBSCAN(eps=0.2, min_samples=5)
labels = dbscan.fit_predict(X)

# Identify outliers (noise points)
outliers = labels == -1

# Plot clusters
plt.figure(figsize=(8, 6))

# Plot clustered points
plt.scatter(X[~outliers, 0],
            X[~outliers, 1],
            c=labels[~outliers],
            cmap='viridis',
            s=50)

# Highlight outliers
plt.scatter(X[outliers, 0],
            X[outliers, 1],
            color='red',
            marker='x',
            s=100,
            label='Outliers')

plt.title('DBSCAN Clustering on make_moons Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```

Output -



24 Load the Wine dataset and apply K-Means clustering after standardizing the features. Print the size of each cluster

```
# Load Wine dataset and apply K-Means clustering

from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import numpy as np

# Load dataset
wine = load_wine()
X = wine.data

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply K-Means
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)
```

```
# Print size of each cluster
unique, counts = np.unique(labels, return_counts=True)

print("Cluster Sizes:")
for cluster, size in zip(unique, counts):
    print(f"Cluster {cluster}: {size} samples")
```

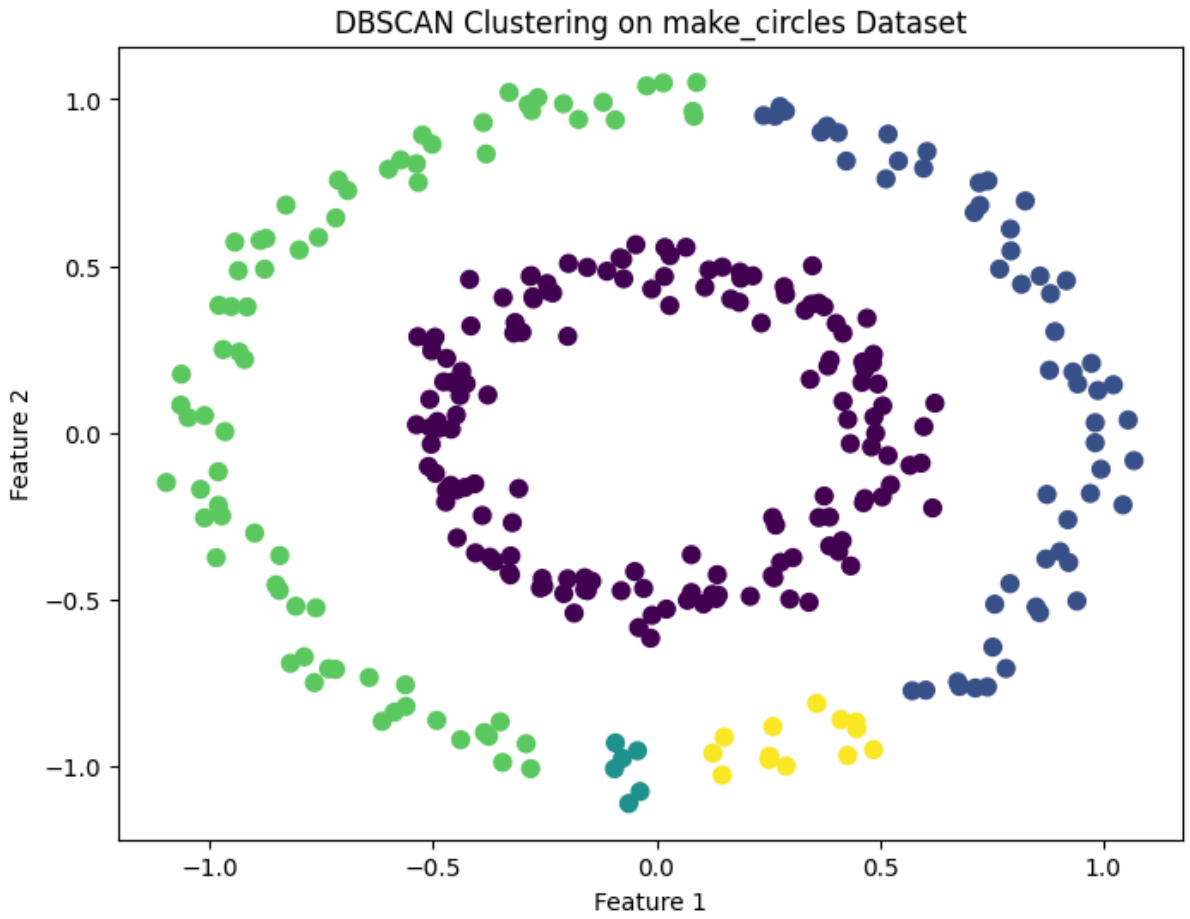
Output -

```
Cluster Sizes:
Cluster 0: 65 samples
Cluster 1: 51 samples
Cluster 2: 62 samples
```

25 Use make_circles to generate synthetic data and cluster it using DBSCAN. Plot the result

```
# Generate synthetic circular data and apply DBSCAN

from sklearn.datasets import make_circles
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

# Generate dataset
X, y = make_circles(n_samples=300,
                    factor=0.5,
                    noise=0.05,
                    random_state=42)

# Apply DBSCAN
dbscan = DBSCAN(eps=0.15, min_samples=5)
labels = dbscan.fit_predict(X)

# Plot the clusters
plt.figure(figsize=(8,6))
plt.scatter(X[:, 0],
            X[:, 1],
            c=labels,
            cmap='viridis',
            s=50)

plt.title('DBSCAN Clustering on make_circles Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Output -



26 Load the Breast Cancer dataset, apply MinMaxScaler, and use K-Means with 2 clusters. Output the cluster centroids

```
# Load Breast Cancer dataset, apply MinMaxScaler,
# and perform K-Means clustering with 2 clusters

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import MinMaxScaler
from sklearn.cluster import KMeans

# Load dataset
cancer = load_breast_cancer()
X = cancer.data

# Apply MinMax Scaling
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Apply K-Means
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
kmeans.fit(X_scaled)
```

```
# Print cluster centroids
print("Cluster Centroids:")
print(kmeans.cluster_centers_)
```

Output -

```
Cluster Centroids:
[[0.50483563 0.39560329 0.50578661 0.36376576 0.46988732 0.42226302
  0.41838662 0.46928035 0.45899738 0.29945886 0.19093085 0.19112073
  0.17903433 0.13086432 0.18017962 0.25890126 0.12542475 0.30942779
  0.190072   0.13266975 0.48047448 0.45107371 0.4655302  0.31460597
  0.49868817 0.36391461 0.39027292 0.65827197 0.33752296 0.26041387]
 [0.25535358 0.28833455 0.24696416 0.14388369 0.35743076 0.18019471
  0.10344776 0.1306603  0.34011829 0.25591606 0.06427485 0.18843043
  0.05975663 0.02870108 0.18158628 0.13242941 0.05821528 0.18069336
  0.17221057 0.08403996 0.2052406  0.32069002 0.19242138 0.09943446
  0.3571115  0.14873935 0.13142287 0.26231363 0.22639412 0.15437354]]
```

27 Generate synthetic data using make_blobs with varying cluster standard deviations and cluster with DBSCAN

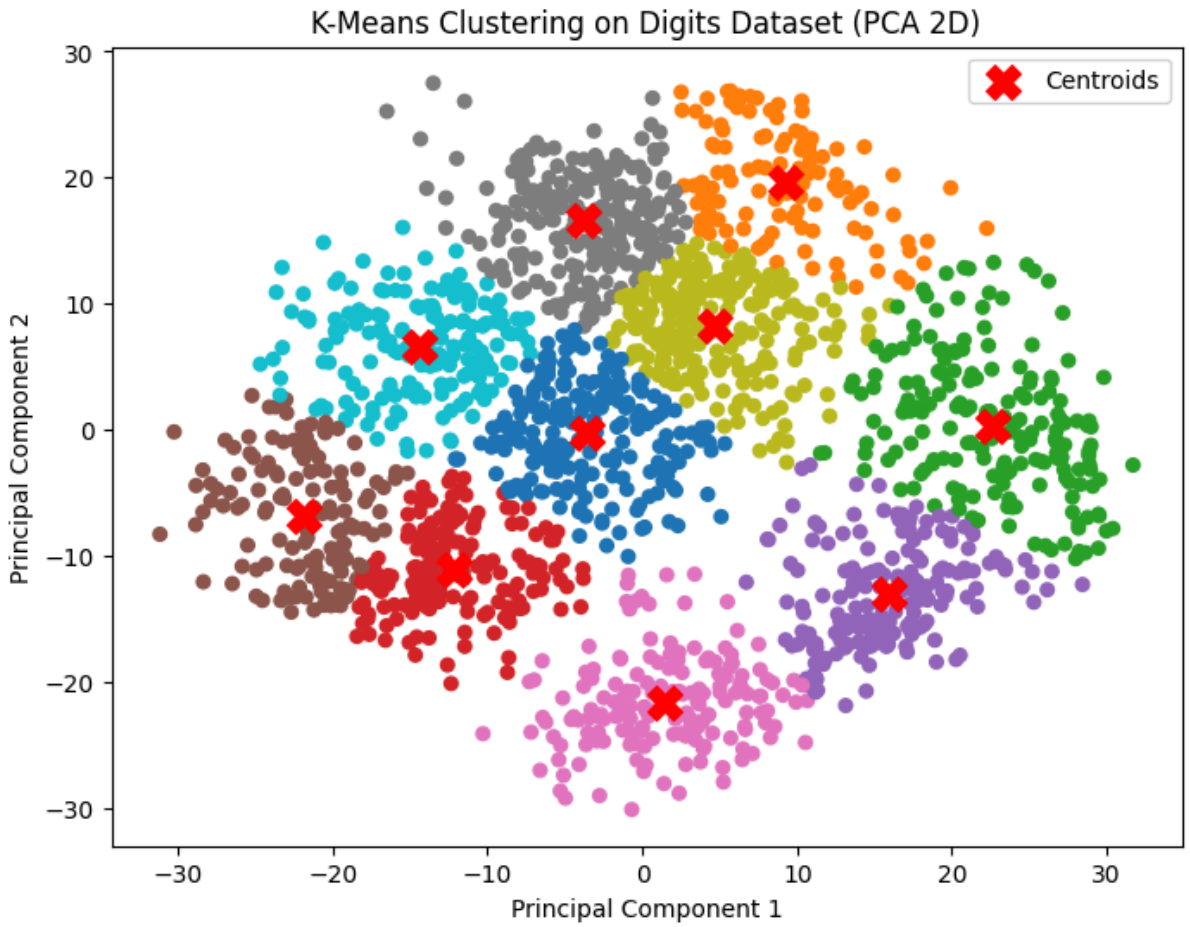
```
# Generate synthetic data with varying cluster standard deviations
# and apply DBSCAN clustering

from sklearn.datasets import make_blobs
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

# Generate dataset
X, y = make_blobs(n_samples=500,
                  centers=3,
                  cluster_std=[0.5, 1.5, 2.5],
                  random_state=42)

# Apply DBSCAN
dbscan = DBSCAN(eps=0.8, min_samples=5)
labels = dbscan.fit_predict(X)

# Plot the clusters
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0],
            X[:, 1],
            c=labels,
            cmap='viridis',
            s=50)

plt.title('DBSCAN Clustering on Blobs with Varying Densities')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.colorbar(label='Cluster Label')
```

```
plt.show()

# Display unique cluster labels
print("Cluster Labels:", set(labels))
```

28 Load the Digits dataset, reduce it to 2D using PCA, and visualize clusters from K-Means

```
# Load Digits dataset, reduce to 2D using PCA,
# and visualize clusters obtained from K-Means

from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Load dataset
digits = load_digits()
X = digits.data

# Reduce dimensions to 2 using PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Apply K-Means clustering
kmeans = KMeans(n_clusters=10, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_pca)

# Plot clusters
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0],
            X_pca[:, 1],
            c=labels,
            cmap='tab10',
            s=30)

# Plot cluster centroids
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            color='red',
            marker='X',
            s=200,
            label='Centroids')

plt.title('K-Means Clustering on Digits Dataset (PCA 2D)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
```

Output -



29 Create synthetic data using make_blobs and evaluate silhouette scores for k = 2 to 5. Display as a bar chart

```
# Generate synthetic data and evaluate Silhouette Scores
# for k = 2 to 5 using K-Means

from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

# Generate synthetic dataset
X, y = make_blobs(n_samples=500,
                  centers=4,
                  cluster_std=1.0,
                  random_state=42)

# Calculate Silhouette Scores
k_values = range(2, 6)
scores = []
```

```
for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X)

    score = silhouette_score(X, labels)
    scores.append(score)

    print(f"k = {k}, Silhouette Score = {score:.4f}")

# Plot Bar Chart
plt.figure(figsize=(8, 5))
plt.bar(k_values, scores)

plt.title('Silhouette Scores for Different Values of k')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')
plt.xticks(k_values)
plt.show()
```

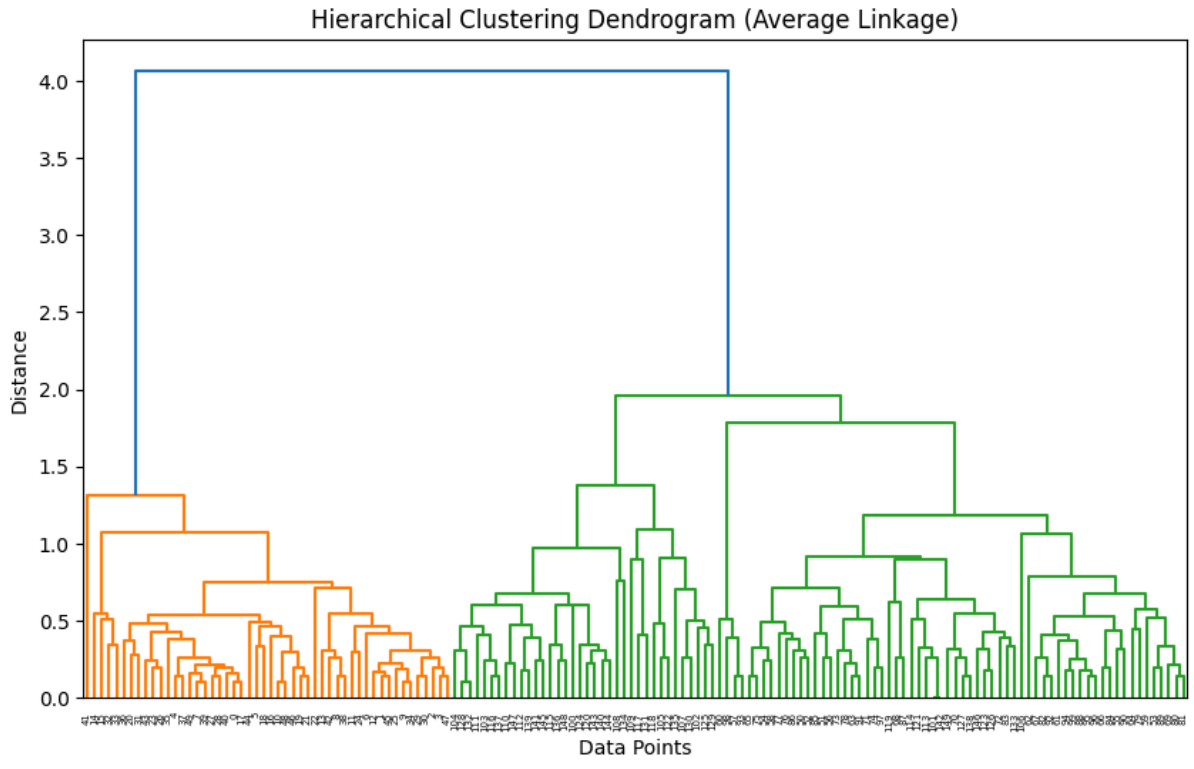
Output -

30 Load the Iris dataset and use hierarchical clustering to group data. Plot a dendrogram with average linkage

```
# Load Iris dataset and perform Hierarchical Clustering
# Plot a dendrogram using Average Linkage
```

```
from sklearn.datasets import load_iris
from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt

# Load dataset
iris = load_iris()
X = iris.data

# Perform hierarchical clustering using average linkage
Z = linkage(X, method='average')

# Plot dendrogram
plt.figure(figsize=(10, 6))
dendrogram(Z)

plt.title('Hierarchical Clustering Dendrogram (Average Linkage)')
plt.xlabel('Data Points')
plt.ylabel('Distance')
plt.show()
```

Output -



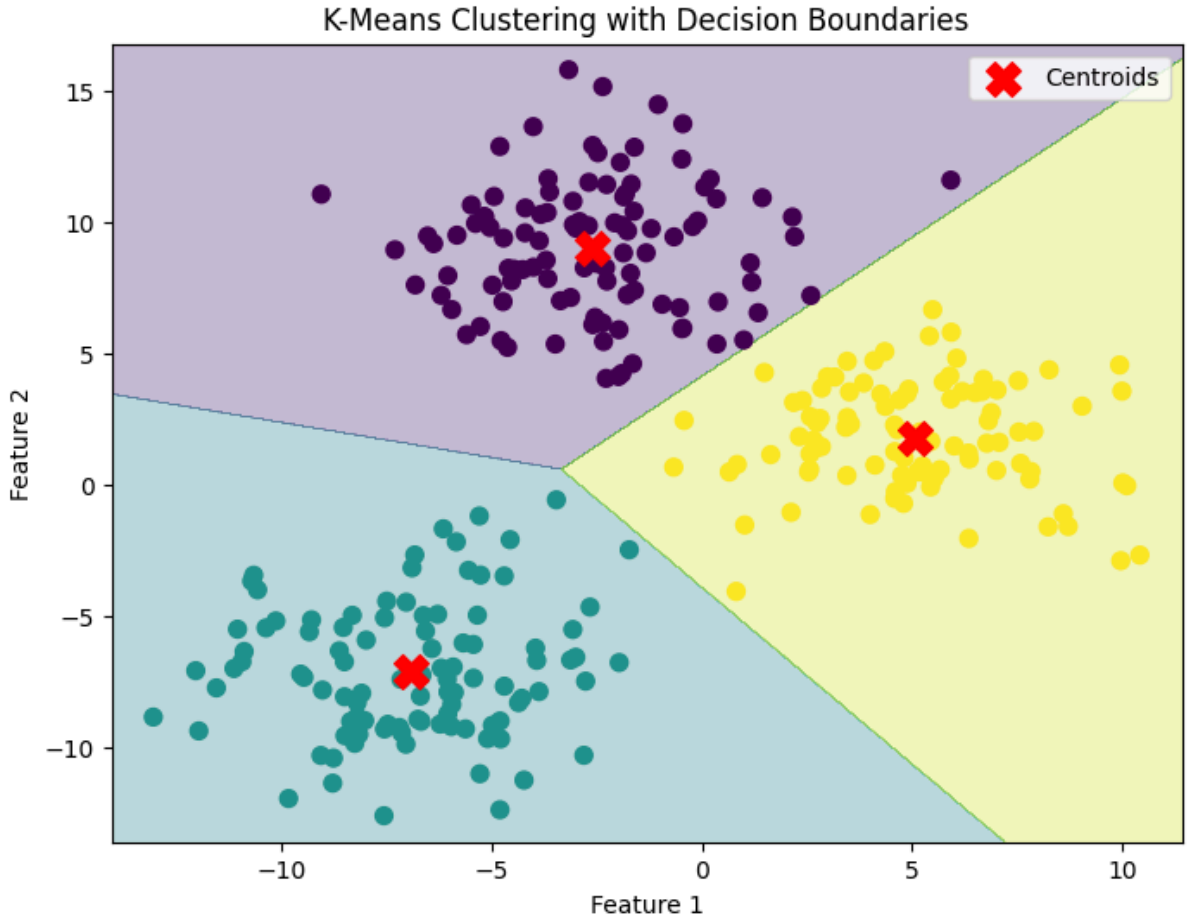
31 Generate synthetic data with overlapping clusters using make_blobs, then apply K-Means and visualize with decision boundaries

```
# Generate overlapping clusters using make_blobs
```

```
# Apply K-Means and visualize decision boundaries

from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
import numpy as np
import matplotlib.pyplot as plt

# Generate overlapping data
X, y = make_blobs(n_samples=300,
                  centers=3,
                  cluster_std=2.5,
                  random_state=42)

# Apply K-Means
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X)

# Create mesh grid for decision boundaries
h = 0.02
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

# Predict cluster for each grid point
Z = kmeans.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot decision boundaries
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='viridis')

# Plot data points
plt.scatter(X[:, 0],
            X[:, 1],
            c=kmeans.labels_,
            cmap='viridis',
            s=50)

# Plot centroids
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            marker='X',
            s=200,
            color='red',
            label='Centroids')
```

```
plt.title('K-Means Clustering with Decision Boundaries')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```

Output -

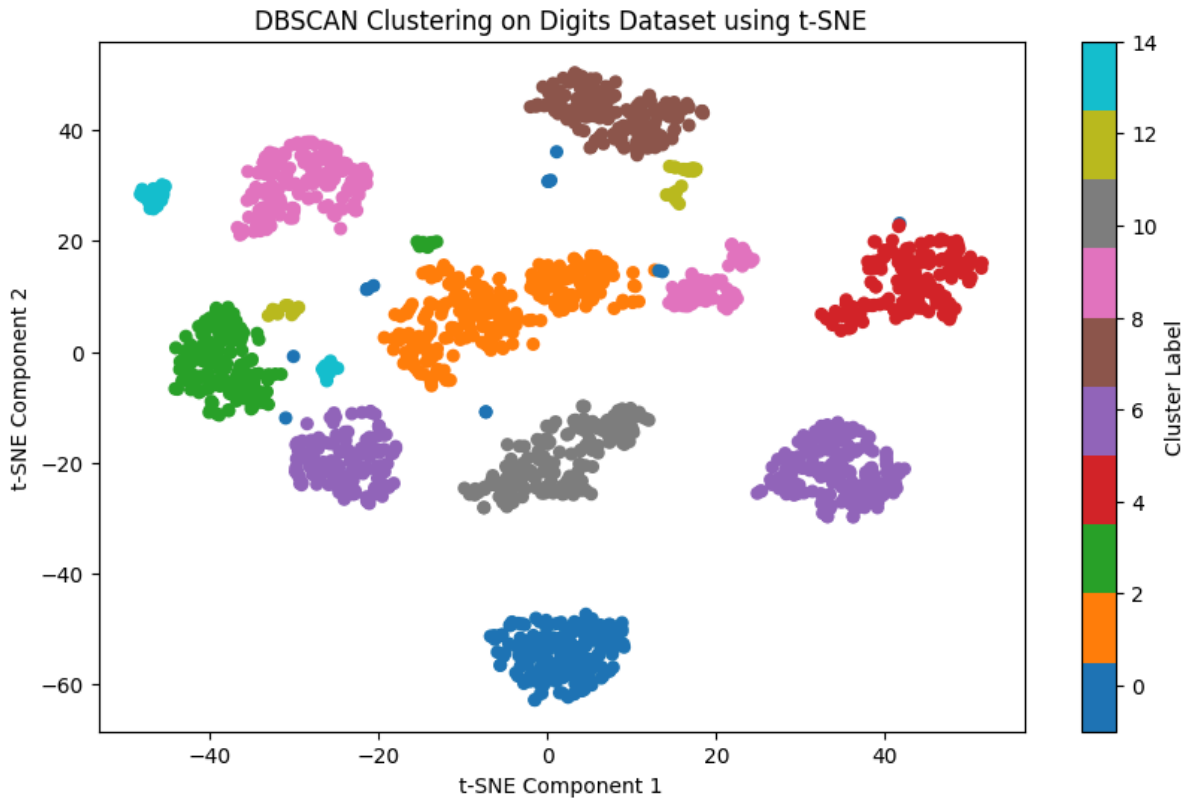


32 Load the Digits dataset and apply DBSCAN after reducing dimensions with t-SNE. Visualize the results

```
# Load Digits dataset, reduce dimensions using t-SNE,
# apply DBSCAN, and visualize the results

from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

# Load dataset
digits = load_digits()
X = digits.data
```

```
# Reduce dimensions to 2D using t-SNE
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)

# Apply DBSCAN
dbscan = DBSCAN(eps=3, min_samples=5)
labels = dbscan.fit_predict(X_tsne)

# Plot clusters
plt.figure(figsize=(10, 6))
scatter = plt.scatter(X_tsne[:, 0],
                      X_tsne[:, 1],
                      c=labels,
                      cmap='tab10',
                      s=30)

plt.title('DBSCAN Clustering on Digits Dataset using t-SNE')
plt.xlabel('t-SNE Component 1')
plt.ylabel('t-SNE Component 2')
plt.colorbar(scatter, label='Cluster Label')
plt.show()

# Display number of clusters
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
print("Number of clusters found:", n_clusters)
```

Output-

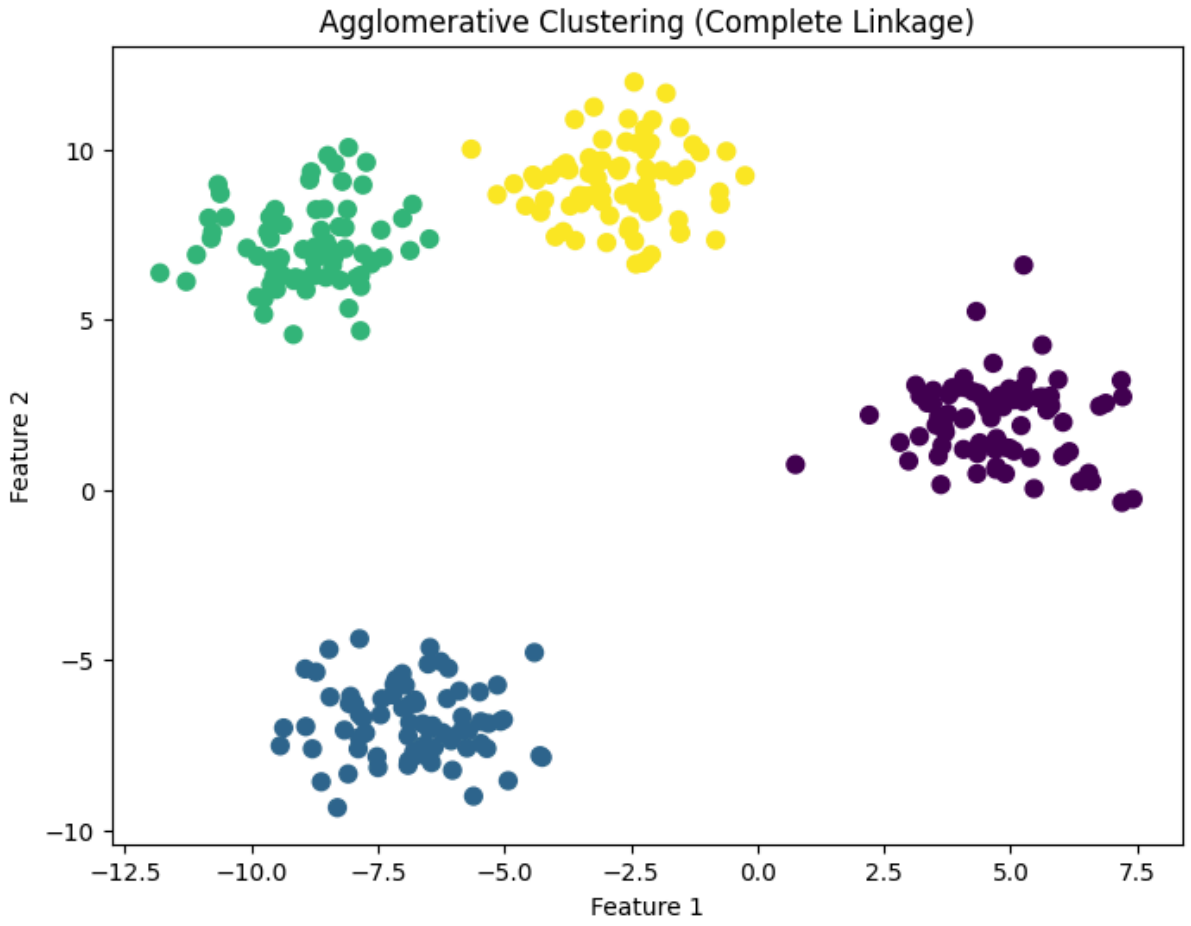


33 Generate synthetic data using make_blobs and apply Agglomerative Clustering with complete linkage. Plot the result

```
# Generate synthetic data using make_blobs
# and apply Agglomerative Clustering with Complete Linkage

from sklearn.datasets import make_blobs
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt

# Generate synthetic dataset
X, y = make_blobs(n_samples=300,
                  centers=4,
                  cluster_std=1.2,
                  random_state=42)

# Apply Agglomerative Clustering with complete linkage
agg = AgglomerativeClustering(
    n_clusters=4,
    linkage='complete'
)

labels = agg.fit_predict(X)

# Visualize clusters
```

```
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0],
            X[:, 1],
            c=labels,
            cmap='viridis',
            s=50)

plt.title('Agglomerative Clustering (Complete Linkage)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Output -

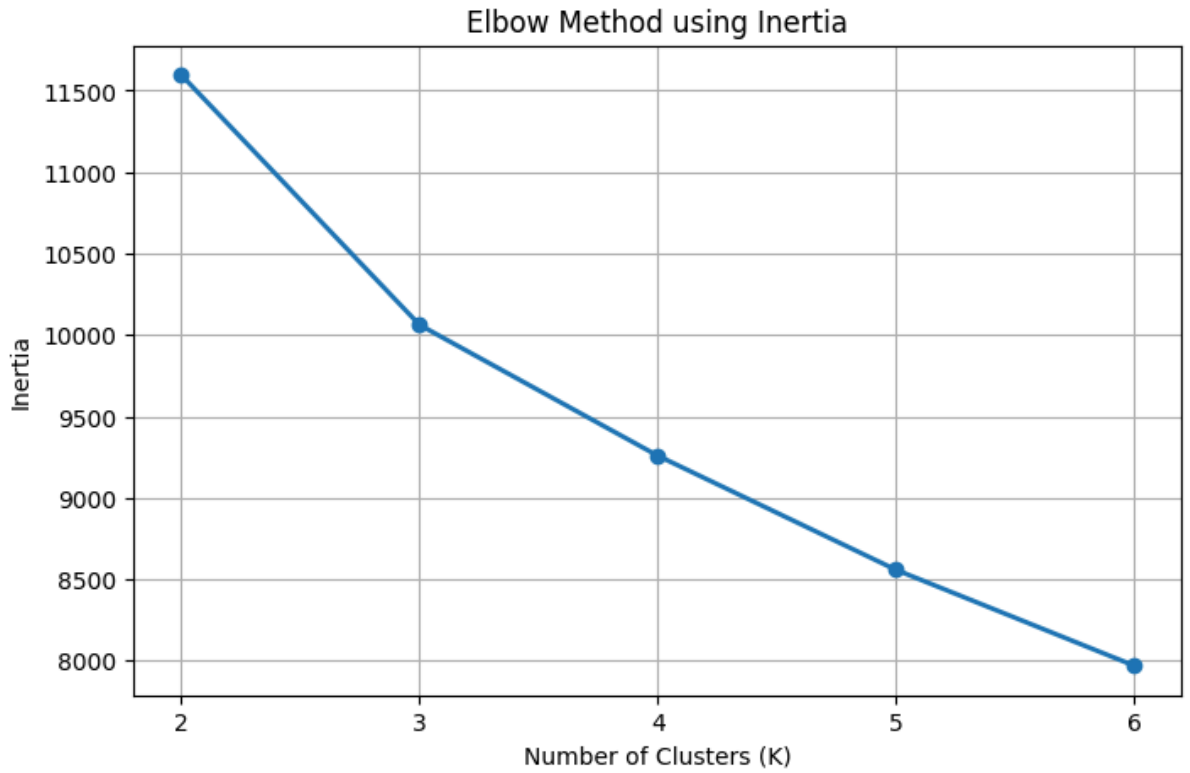


34 Load the Breast Cancer dataset and compare inertia values for K = 2 to 6 using K-Means. Show results in a line plot

```
# Load Breast Cancer dataset and compare inertia values
# for K = 2 to 6 using K-Means

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

```
import matplotlib.pyplot as plt

# Load dataset
cancer = load_breast_cancer()
X = cancer.data

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Calculate inertia for different K values
k_values = range(2, 7)
inertia_values = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia_values.append(kmeans.inertia_)

    print(f"K = {k}, Inertia = {kmeans.inertia_:.2f}")

# Plot inertia values
plt.figure(figsize=(8, 5))
plt.plot(k_values,
         inertia_values,
         marker='o',
         linewidth=2)

plt.title('Elbow Method using Inertia')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.xticks(k_values)
plt.grid(True)
plt.show()
```

Output-

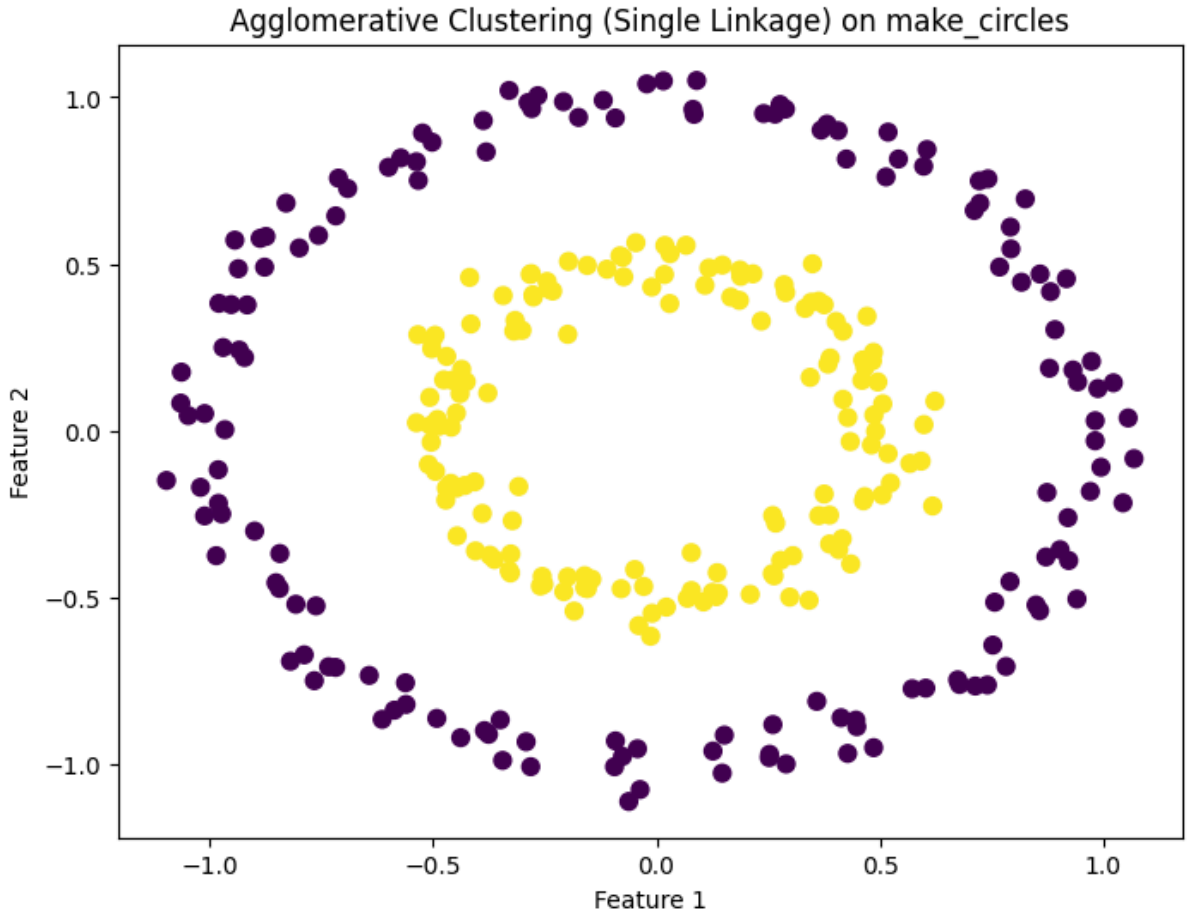


35 Generate synthetic concentric circles using make_circles and cluster using Agglomerative Clustering with single linkage

```
# Generate synthetic concentric circles and apply
# Agglomerative Clustering with Single Linkage

from sklearn.datasets import make_circles
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt

# Generate concentric circles dataset
X, y = make_circles(n_samples=300,
                    factor=0.5,
                    noise=0.05,
                    random_state=42)

# Apply Agglomerative Clustering with single linkage
agg = AgglomerativeClustering(
    n_clusters=2,
    linkage='single'
)

labels = agg.fit_predict(X)

# Plot the clusters
plt.figure(figsize=(8, 6))
```

```
plt.scatter(X[:, 0],
            X[:, 1],
            c=labels,
            cmap='viridis',
            s=50)

plt.title('Agglomerative Clustering (Single Linkage) on make_circles')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Output -



36 Use the Wine dataset, apply DBSCAN after scaling the data, and count the number of clusters (excluding noise

```
# Load Wine dataset, scale the features,
# apply DBSCAN, and count clusters excluding noise

from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN
```

```
# Load dataset
wine = load_wine()
X = wine.data

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply DBSCAN
dbscan = DBSCAN(eps=2.5, min_samples=5)
labels = dbscan.fit_predict(X_scaled)

# Count clusters excluding noise (-1)
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

print("Number of clusters (excluding noise):", n_clusters)

# Optional: Display cluster labels
print("Unique labels:", set(labels))
```

Output-

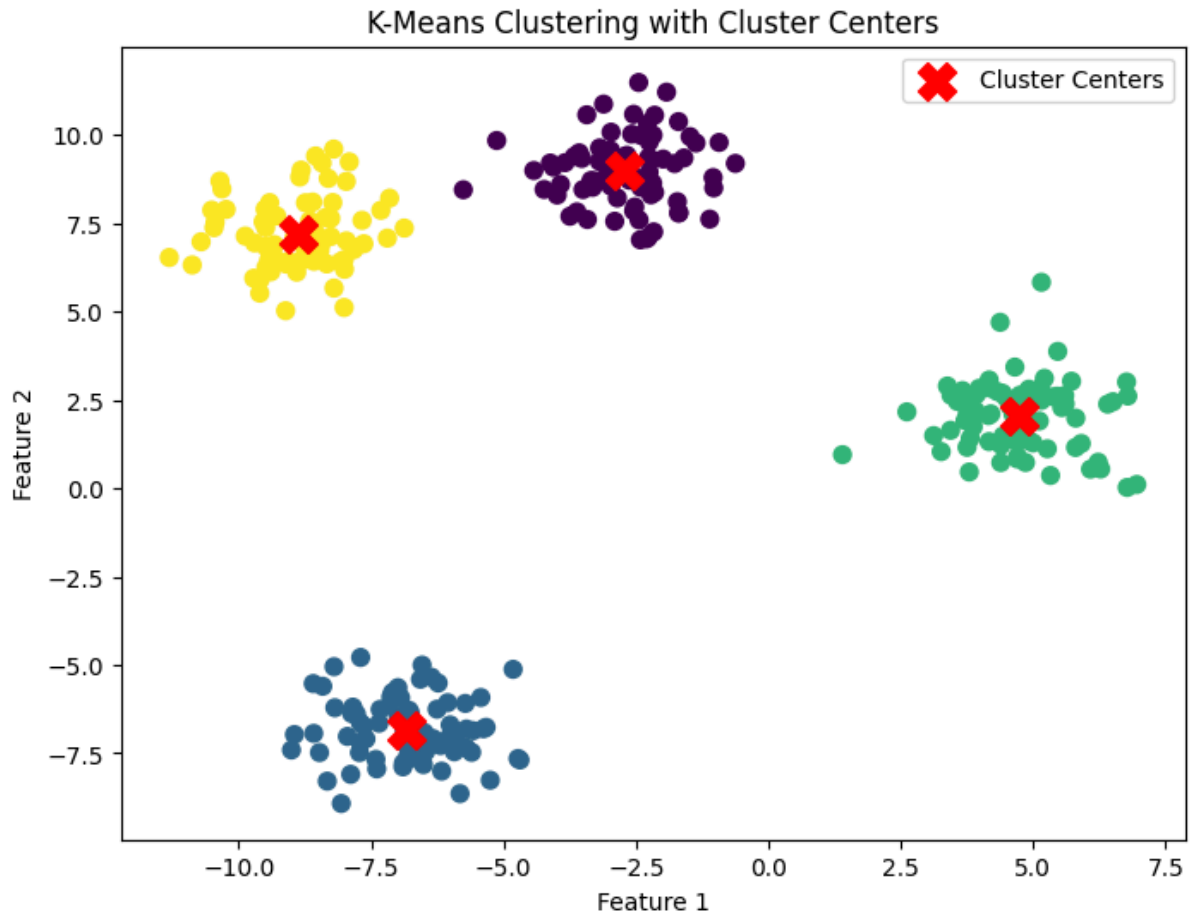
```
Number of clusters (excluding noise): 1
Unique labels: {np.int64(0), np.int64(-1)}
```

37 Generate synthetic data with make_blobs and apply KMeans. Then plot the cluster centers on top of the data points

```
# Generate synthetic data using make_blobs
# Apply K-Means and plot cluster centers

from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Generate synthetic dataset
X, y = make_blobs(n_samples=300,
                  centers=4,
                  cluster_std=1.0,
                  random_state=42)

# Apply K-Means clustering
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
labels = kmeans.fit_predict(X)

# Plot data points
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0],
            X[:, 1],
```

```
            c=labels,
            cmap='viridis',
            s=50)

# Plot cluster centers
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            marker='X',
            color='red',
            s=250,
            label='Cluster Centers')

plt.title('K-Means Clustering with Cluster Centers')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```

Output -



38 Load the Iris dataset, cluster with DBSCAN, and print how many samples were identified as noise

```
# Load Iris dataset, apply DBSCAN,
# and count the number of noise samples

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN

# Load dataset
iris = load_iris()
X = iris.data

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply DBSCAN
dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(X_scaled)

# Count noise samples (-1 label)
noise_count = sum(labels == -1)

print("Number of samples identified as noise:", noise_count)
```

Output -

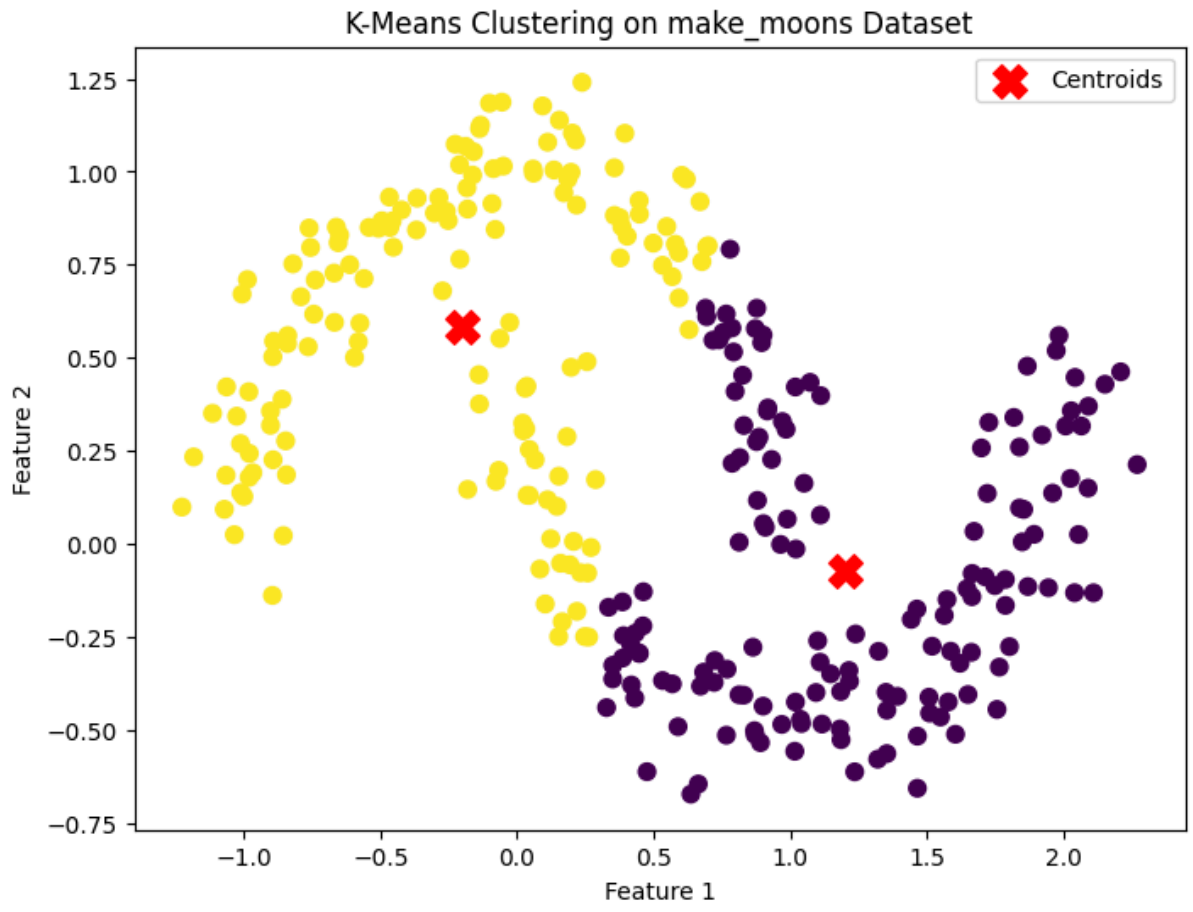
```
Number of samples identified as noise: 34
```

39 Generate synthetic non-linearly separable data using make_moons, apply K-Means, and visualize the clustering result

```
# Generate non-linearly separable data using make_moons
# Apply K-Means clustering and visualize the result

from sklearn.datasets import make_moons
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Generate dataset
X, y = make_moons(n_samples=300,
                  noise=0.1,
                  random_state=42)

# Apply K-Means
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
labels = kmeans.fit_predict(X)

# Plot clustering result
```

```
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0],
            X[:, 1],
            c=labels,
            cmap='viridis',
            s=50)

# Plot cluster centers
plt.scatter(kmeans.cluster_centers_[:, 0],
            kmeans.cluster_centers_[:, 1],
            color='red',
            marker='X',
            s=200,
            label='Centroids')

plt.title('K-Means Clustering on make_moons Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```

Output -

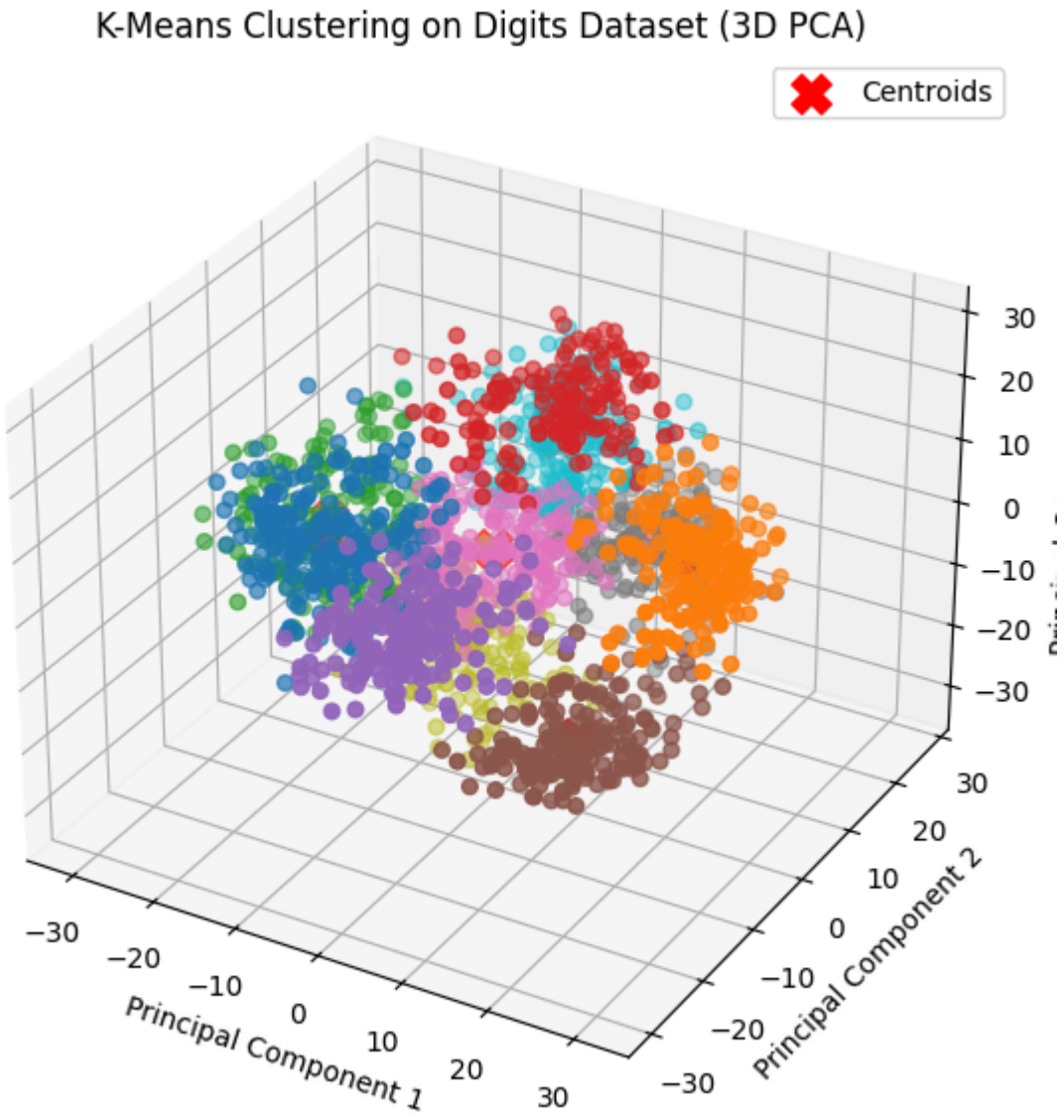


40 Load the Digits dataset, apply PCA to reduce to 3 components, then use KMeans and visualize with a 3D scatter plot

```
# Load Digits dataset, reduce to 3 dimensions using PCA,
# apply K-Means, and visualize with a 3D scatter plot

from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Load dataset
digits = load_digits()
X = digits.data

# Reduce dimensions to 3 using PCA
pca = PCA(n_components=3)
X_pca = pca.fit_transform(X)

# Apply K-Means clustering
kmeans = KMeans(n_clusters=10, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_pca)

# Create 3D scatter plot
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    X_pca[:, 2],
    c=labels,
    cmap='tab10',
    s=30
)

# Plot cluster centroids
ax.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    kmeans.cluster_centers_[:, 2],
    color='red',
    marker='X',
    s=200,
    label='Centroids'
)

ax.set_title('K-Means Clustering on Digits Dataset (3D PCA)')
```

```
ax.set_xlabel('Principal Component 1')
ax.set_ylabel('Principal Component 2')
ax.set_zlabel('Principal Component 3')

plt.legend()
plt.show()
```

Output -



41 Generate synthetic blobs with 5 centers and apply KMeans. Then use silhouette_score to evaluate the clustering

```
# Generate synthetic data with 5 centers,
# apply K-Means, and evaluate using Silhouette Score

from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
```

```
from sklearn.metrics import silhouette_score

# Generate synthetic dataset
X, y = make_blobs(n_samples=500,
                  centers=5,
                  cluster_std=1.0,
                  random_state=42)

# Apply K-Means
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
labels = kmeans.fit_predict(X)

# Calculate Silhouette Score
score = silhouette_score(X, labels)

print("Silhouette Score:", round(score, 4))
```

Output -

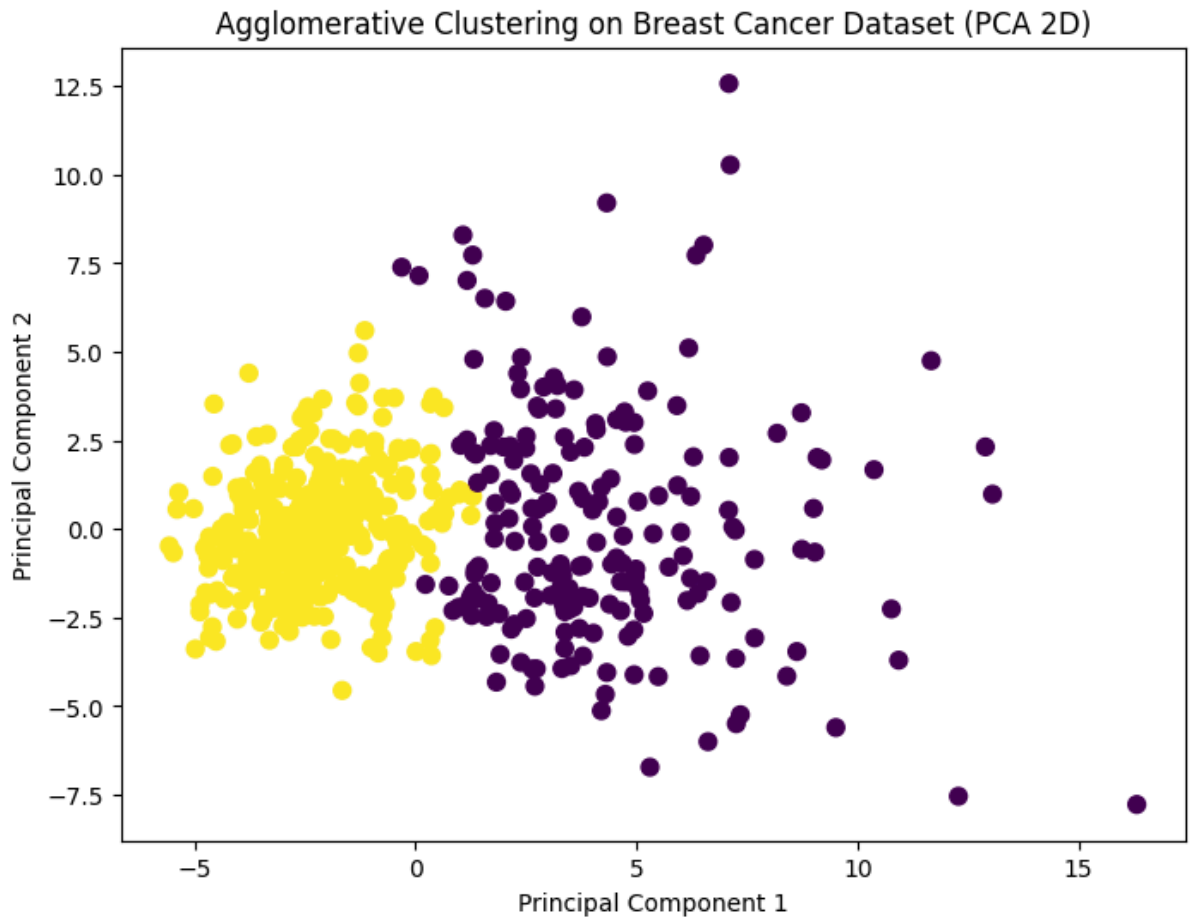
```
Silhouette Score: 0.6786
```

42 Load the Breast Cancer dataset, reduce dimensionality using PCA, and apply Agglomerative Clustering. Visualize in 2D

```
# Load Breast Cancer dataset, reduce dimensions using PCA,
# apply Agglomerative Clustering, and visualize in 2D

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt

# Load dataset
cancer = load_breast_cancer()
X = cancer.data

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Reduce dimensions to 2 using PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
# Apply Agglomerative Clustering
agg = AgglomerativeClustering(n_clusters=2)
labels = agg.fit_predict(X_pca)

# Visualize clusters
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0],
            X_pca[:, 1],
            c=labels,
            cmap='viridis',
            s=50)

plt.title('Agglomerative Clustering on Breast Cancer Dataset (PCA 2D)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```

Output -

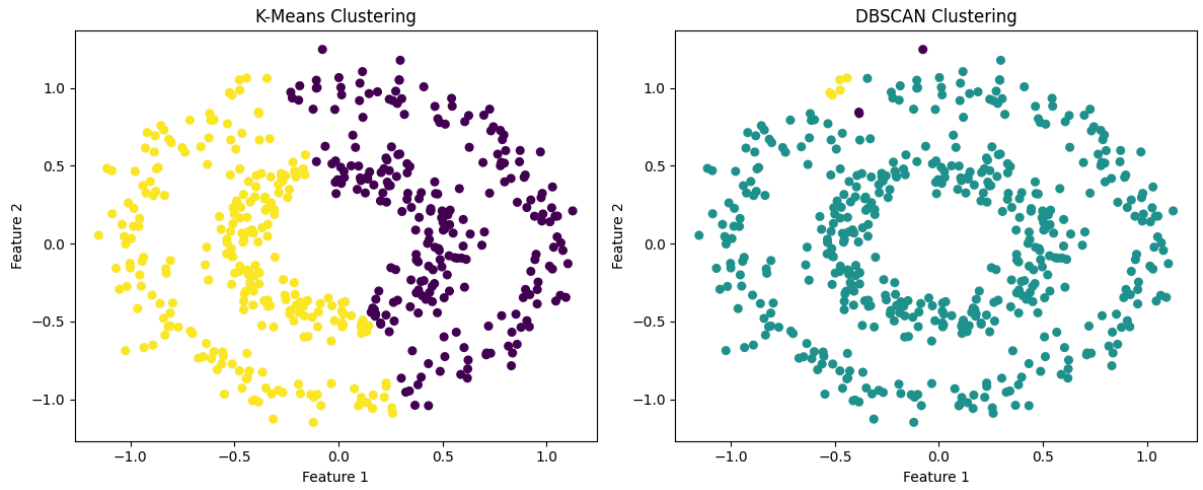


43 Generate noisy circular data using make_circles and visualize clustering results from KMeans and DBSCAN side-by-side

```
# Generate noisy circular data and compare
# K-Means vs DBSCAN clustering side-by-side

from sklearn.datasets import make_circles
from sklearn.cluster import KMeans, DBSCAN
import matplotlib.pyplot as plt

# Generate noisy circular dataset
X, y = make_circles(n_samples=500,
                    factor=0.5,
                    noise=0.08,
                    random_state=42)

# Apply K-Means
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X)

# Apply DBSCAN
dbscan = DBSCAN(eps=0.15, min_samples=5)
dbscan_labels = dbscan.fit_predict(X)

# Create side-by-side plots
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# K-Means result
axes[0].scatter(X[:, 0],
                X[:, 1],
                c=kmeans_labels,
                cmap='viridis',
                s=30)
axes[0].set_title('K-Means Clustering')
axes[0].set_xlabel('Feature 1')
axes[0].set_ylabel('Feature 2')

# DBSCAN result
axes[1].scatter(X[:, 0],
                X[:, 1],
                c=dbscan_labels,
                cmap='viridis',
```

```
            s=30)
axes[1].set_title('DBSCAN Clustering')
axes[1].set_xlabel('Feature 1')
axes[1].set_ylabel('Feature 2')

plt.tight_layout()
plt.show()
```

Output -



44 Load the Iris dataset and plot the Silhouette Coefficient for each sample after KMeans clustering

```
# Load Iris dataset, apply K-Means clustering,
# compute Silhouette Coefficient for each sample,
# and plot the results

from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_samples, silhouette_score
import matplotlib.pyplot as plt
import numpy as np

# Load dataset
iris = load_iris()
X = iris.data

# Apply K-Means
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(X)
```

```
# Compute silhouette scores
sample_silhouette_values = silhouette_samples(X, cluster_labels)
avg_score = silhouette_score(X, cluster_labels)

# Plot silhouette coefficients
plt.figure(figsize=(8, 6))

y_lower = 10
for i in range(3):
    ith_cluster_values = sample_silhouette_values[cluster_labels == i]
    ith_cluster_values.sort()

    size_cluster_i = ith_cluster_values.shape[0]
    y_upper = y_lower + size_cluster_i

    plt.fill_betweenx(
        np.arange(y_lower, y_upper),
        0,
        ith_cluster_values,
        alpha=0.7
    )

    plt.text(-0.05, y_lower + 0.5 * size_cluster_i, str(i))
    y_lower = y_upper + 10

# Average silhouette score line
plt.axvline(x=avg_score, color='red', linestyle='--',
            label=f'Average Score = {avg_score:.3f}')

plt.title('Silhouette Plot for Iris Dataset (K-Means)')
plt.xlabel('Silhouette Coefficient')
plt.ylabel('Cluster')
plt.legend()
plt.show()

print("Average Silhouette Score:", round(avg_score, 4))
```

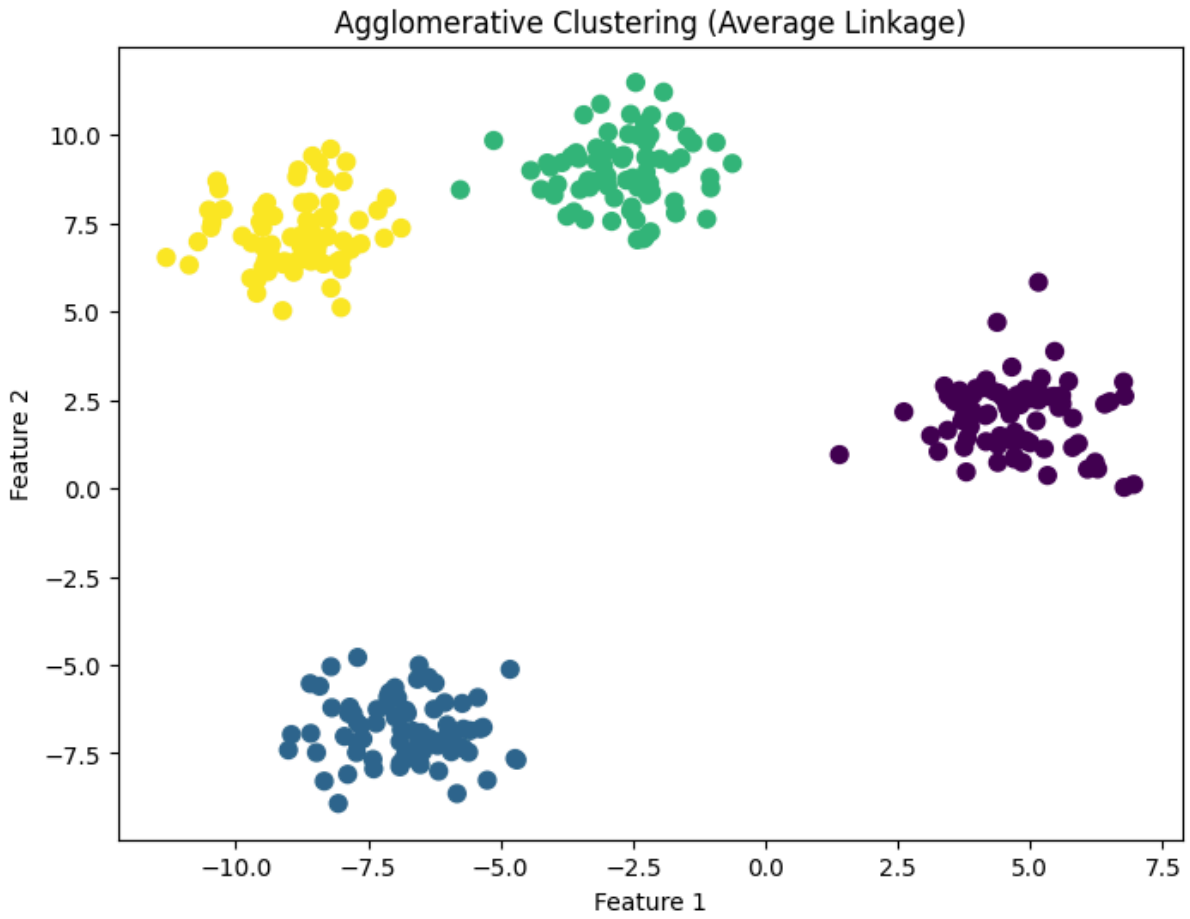
Output -

```
Average Silhouette Score: 0.5528
```

45 Generate synthetic data using make_blobs and apply Agglomerative Clustering with 'average' linkage. Visualize clusters

```
# Generate synthetic data using make_blobs
# and apply Agglomerative Clustering with Average Linkage

from sklearn.datasets import make_blobs
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt

# Generate synthetic dataset
X, y = make_blobs(n_samples=300,
                  centers=4,
                  cluster_std=1.0,
                  random_state=42)

# Apply Agglomerative Clustering with average linkage
agg = AgglomerativeClustering(
    n_clusters=4,
    linkage='average'
)

labels = agg.fit_predict(X)

# Visualize clusters
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0],
            X[:, 1],
            c=labels,
            cmap='viridis',
            s=50)

plt.title('Agglomerative Clustering (Average Linkage)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Output -

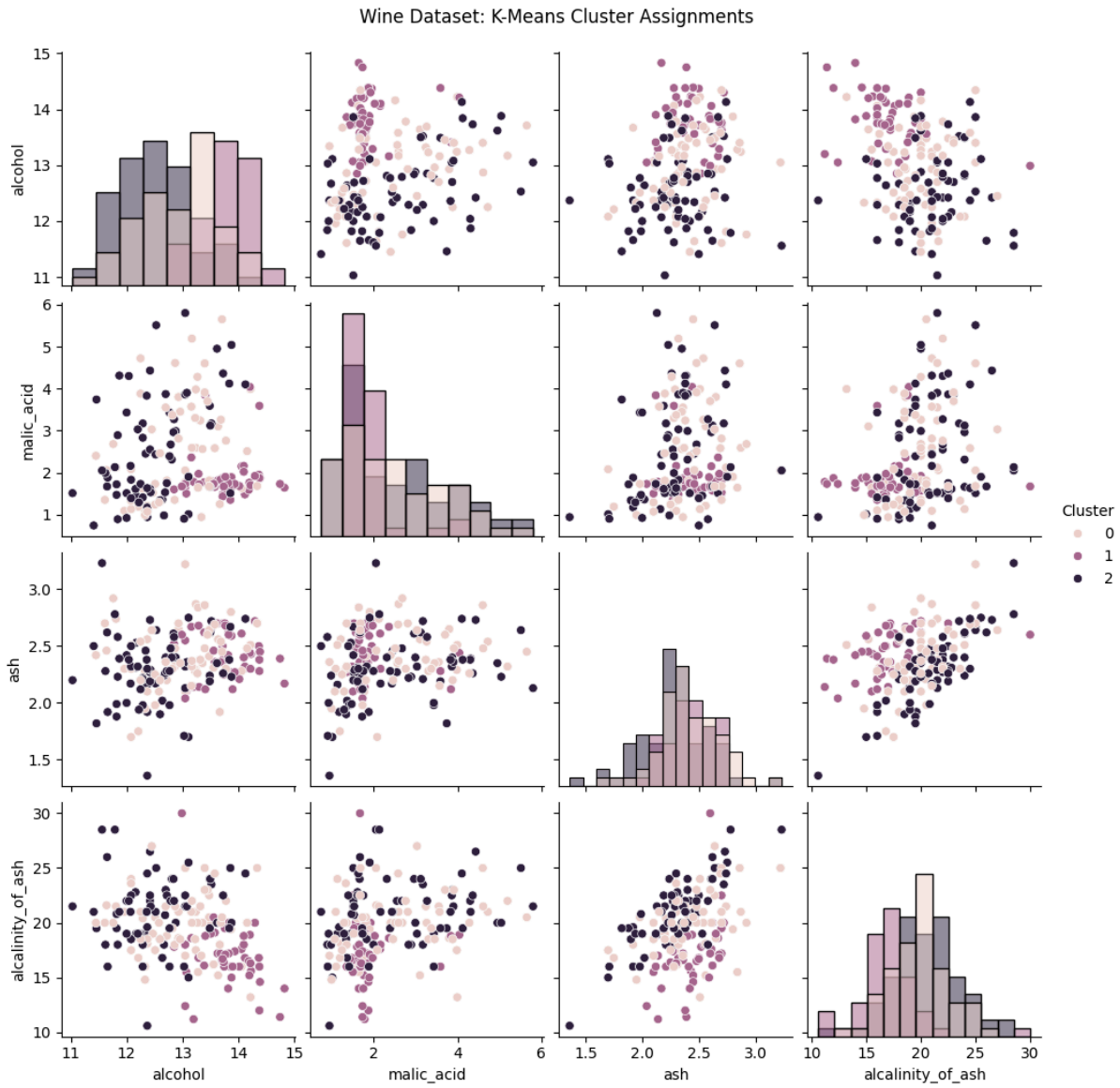


46 Load the Wine dataset, apply KMeans, and visualize the cluster assignments in a seaborn pairplot (first 4 features)

```
# Load Wine dataset, apply K-Means clustering,
# and visualize cluster assignments using a Seaborn pairplot

from sklearn.datasets import load_wine
from sklearn.cluster import KMeans
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
wine = load_wine()

# Create DataFrame with first 4 features
df = pd.DataFrame(
    wine.data[:, :4],
    columns=wine.feature_names[:4]
```

```
)

# Apply K-Means
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(wine.data)

# Create pairplot
sns.pairplot(
    df,
    vars=wine.feature_names[:4],
    hue='Cluster',
    diag_kind='hist'
)

plt.suptitle('Wine Dataset: K-Means Cluster Assignments', y=1.02)
plt.show()
```

Output -



47 Generate noisy blobs using make_blobs and use DBSCAN to identify both clusters and noise points. Print the count

Generate noisy blobs, apply DBSCAN,

and print the number of clusters and noise points

```
from sklearn.datasets import make_blobs
from sklearn.cluster import DBSCAN
import numpy as np

# Generate synthetic data
X, y = make_blobs(n_samples=500,
                  centers=4,
                  cluster_std=1.2,
                  random_state=42)
```

```
# Add some random noise points
noise = np.random.uniform(low=-15, high=15, size=(50, 2))
X = np.vstack([X, noise])

# Apply DBSCAN
dbscan = DBSCAN(eps=0.8, min_samples=5)
labels = dbscan.fit_predict(X)

# Count clusters (excluding noise)
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

# Count noise points
n_noise = np.sum(labels == -1)

print("Number of Clusters:", n_clusters)
print("Number of Noise Points:", n_noise)
```

Output -

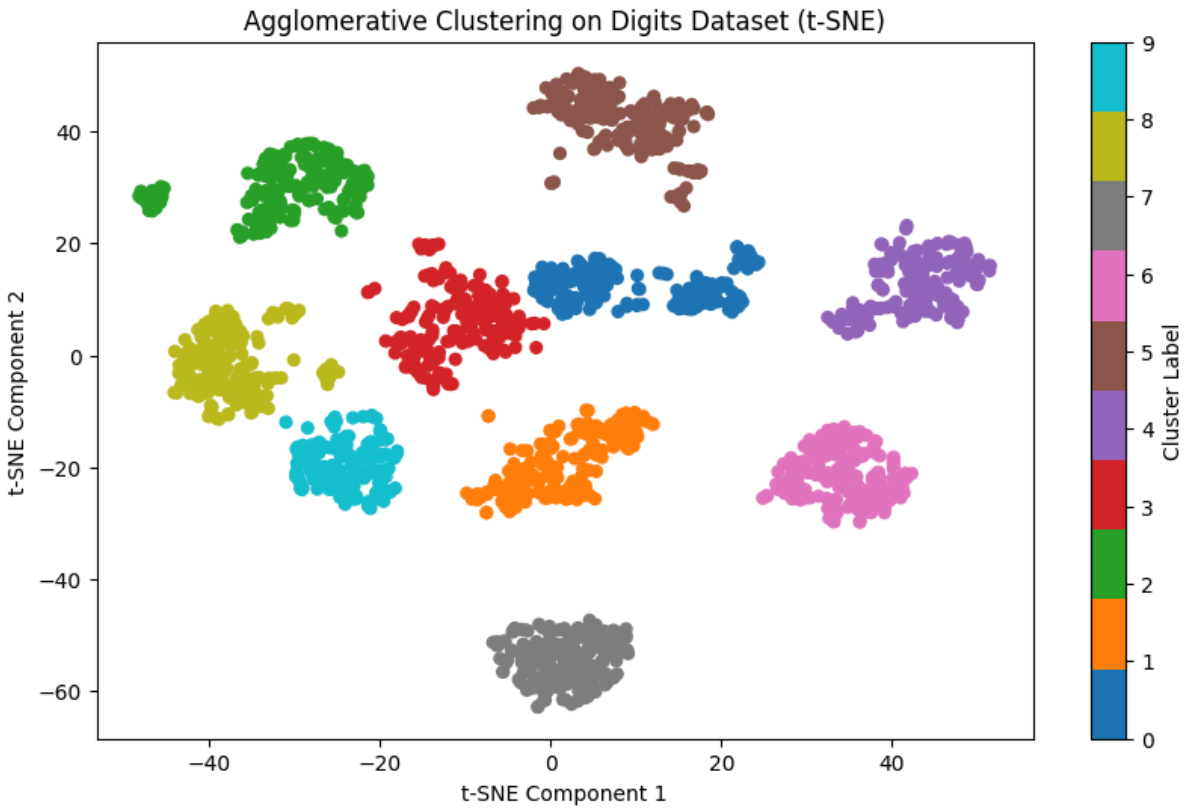
```
Number of Clusters: 4
Number of Noise Points: 74
```

48 Load the Digits dataset, reduce dimensions using t-SNE, then apply Agglomerative Clustering and plot the clusters.

```
# Load Digits dataset, reduce dimensions using t-SNE,
# apply Agglomerative Clustering, and visualize the clusters

from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt

# Load dataset
digits = load_digits()
X = digits.data

# Reduce dimensions to 2D using t-SNE
tsne = TSNE(
    n_components=2,
    random_state=42,
    perplexity=30
)
```

```
X_tsne = tsne.fit_transform(X)

# Apply Agglomerative Clustering
agg = AgglomerativeClustering(
    n_clusters=10,
    linkage='ward'
)

labels = agg.fit_predict(X_tsne)

# Visualize clusters
plt.figure(figsize=(10, 6))
plt.scatter(
    X_tsne[:, 0],
    X_tsne[:, 1],
    c=labels,
    cmap='tab10',
    s=30
)

plt.title('Agglomerative Clustering on Digits Dataset (t-SNE)')
plt.xlabel('t-SNE Component 1')
plt.ylabel('t-SNE Component 2')
plt.colorbar(label='Cluster Label')
plt.show()
```

Output -


Agglomerative Clustering on Digits Dataset (t-SNE)
t-SNE Component 1
t-SNE Component 2
Cluster Label
40
20
0
−20
−40
−60
−40
−20
0
20
40
9
8
7
6
5
4
3
2
1
0